IP Core Testbench, Annotated

`tb_signalchain_ip.sv` for the HDL Coder IP core `SignalCha_ip`

Purpose

This testbench verifies the HDL Coder IP core `SignalCha_ip` on its own before it is placed in the Zynq block design. It configures the IP over AXI4-Lite, receives its AXI4-Stream output with deliberate backpressure, and logs every beat for comparison with the HDL cosimulation.

The AXI handshaking is written by hand on purpose: to learn the AXI4-Lite and AXI4-Stream protocols at the signal level, so that every rule in the specifications [1, 2] maps to specific lines of code. Ready-made alternatives exist, notably MathWorks HDL Verifier's AXI Manager simulation tasks [12], which are built on the AMD AXI Verification IP [9]; in a production flow those would be preferred for their built-in protocol checking.

The eleven rules

Annotations refer to these rules by number.

#	Rule	Channel
1	Data moves only on a rising edge where VALID and READY are both high	Lite, Stream
2	A source raises VALID without waiting for READY	Lite
3	Once VALID is high, VALID and its data hold until the transfer	Lite
4	A destination may raise READY before VALID and drop it at any time	Lite, Stream
5	Write address (AW) and write data (W) are independent channels	Lite write
6	A write is not finished until its B response arrives	Lite write
7	A read is an AR handshake, then an R handshake	Lite read
8	Every response carries a status; 0 means OKAY	Lite
9	Stream data counts only on a transfer edge	Stream
10	TLAST marks the last beat of a packet and travels with it	Stream
11	Nothing starts until reset is released	All

Reading the timing

Construct	Meaning
`@(posedge clk)`	Wait for the next rising clock edge.
reading a signal after the edge	Gives its value from just before the edge: what a flip-flop captures.
`x <= v;`	Nonblocking: takes effect after the edge, so the IP sees it at the next edge.
`do @(posedge clk); while (!rdy);`	Check READY at each edge; exit on the edge where it was high, which is the transfer.
`fork ... join`	Run branches in parallel; continue once all have finished.

Edge numbers count rising clock edges from the start of the simulation, 8 ns apart. The testbench's `cycle` counter holds the same number. Language details follow IEEE 1800 [3].

The testbench, annotated

```
1  // IP-level testbench for the HDL Coder IP core SignalCha_ip.
2  // Reads/writes the AXI4-Lite registers, receives the AXI4-Stream
       output with
3  // random backpressure plus one long stall (fills the IP's output FIFO
       ), and
4  // writes every accepted beat as "TDATA TLAST" to the file given by +
       out=<file>.
5
6  `timescale 1ns / 1ps
7
8  module tb_signalchain_ip;
```

Lines 1–8: File header, time unit, module

`` `timescale 1ns / 1ps `` sets the unit for every delay in this file: `#4` means 4 ns, and times are resolved to 1 ps [3]. The testbench is a module with no ports; everything it needs is declared inside. The header comment states the three jobs: drive the AXI4-Lite registers, receive the AXI4-Stream output with backpressure, and log every accepted beat.

```
9
10   localparam int FRAME    = 1024;
11   localparam int N_FRAMES = 2;
12
13   // register offsets (ipcore/SignalCha_ip_v1_0/include/
       SignalCha_ip_addr.h)
14   localparam logic [15:0] REG_PACKET_SIZE = 16'h0008;
15   localparam logic [15:0] REG_TIMESTAMP   = 16'h000C;
16   localparam logic [15:0] REG_ENABLE      = 16'h0100;
```

Lines 9–16: Constants and the IP register map

`FRAME` and `N_FRAMES` set the test length: two 1024-sample FFT frames. The three register offsets come from the C header HDL Coder generates with the IP core (`SignalCha_ip_addr.h`) [10]:

`0x008` packet size (sets TLAST), `0x00C` build timestamp, `0x100` the model's `enable` input. Offsets are byte addresses, 4-byte aligned, as AXI4-Lite requires for 32-bit data [1].

```
17
18  logic clk = 0, rstn = 0;
19  always #4 clk = ~clk;    // 125 MHz
```

Lines 17–19: Clock and reset

`always #4 clk = ~clk;` toggles every 4 ns: an 8 ns period, 125 MHz, the same as the Zynq FCLK0 the IP will run on. `rstn` is the active-low reset (AXI's ARESETn convention [1]) and starts at 0, so the IP begins in reset. **Rule 11**: nothing may transfer until reset is released.

```
20
21  // AXI4-Lite
22  logic [15:0] awaddr = 0, araddr = 0;
23  logic [31:0] wdata = 0, rdata;
24  logic        awvalid = 0, wvalid = 0, bready = 0, arvalid = 0,
     rready = 0;
25  logic        awready, wready, bvalid, arready, rvalid;
26  logic [1:0]  bresp, rresp;
27
28  // AXI4-Stream
29  logic [31:0] tdata;
30  logic        tvalid, tlast, tready = 0;
```

Lines 20–31: AXI4-Lite and AXI4-Stream signals

One group of wires per AXI channel [1]: **AW** (write address), **W** (write data), **B** (write response), **AR** (read address), **R** (read data); and the stream signals TDATA, TVALID, TLAST, TREADY [2]. Every signal the testbench *drives* is initialised to 0 (`awvalid`, `wvalid`, `bready`, `arvalid`, `rready`, `tready`), so all VALIDs and READYs are low during reset (**rule 11**). Signals the IP drives (`awready`, `rdata`, `tvalid`, ...) are left uninitialised; the IP sets them. `logic` is SystemVerilog's 4-state variable type [3, 4].

```
32  SignalCha_ip dut (
33    .IPCORE_CLK               (clk),
34    .IPCORE_RESETN            (rstn),
35    .AXI4_Lite_ACLK           (clk),
36    .AXI4_Lite_ARESETN        (rstn),
37    .AXI4_Lite_AWADDR         (awaddr),
38    .AXI4_Lite_AWVALID        (awvalid),
39    .AXI4_Lite_WDATA          (wdata),
40    .AXI4_Lite_WSTRB          (4'hF),
41    .AXI4_Lite_WVALID         (wvalid),
42    .AXI4_Lite_BREADY         (bready),
43    .AXI4_Lite_ARADDR         (araddr),
44    .AXI4_Lite_ARVALID        (arvalid),
45    .AXI4_Lite_RREADY         (rready),
46    .AXI4_Lite_AWREADY        (awready),
```

```
47      .AXI4_Lite_WREADY           (wready),
48      .AXI4_Lite_BRESP            (bresp),
49      .AXI4_Lite_BVALID           (bvalid),
50      .AXI4_Lite_ARREADY          (arready),
51      .AXI4_Lite_RDATA            (rdata),
52      .AXI4_Lite_RRESP            (rresp),
53      .AXI4_Lite_RVALID           (rvalid),
54      .AXI4_Stream_Master_TREADY (tready),
55      .AXI4_Stream_Master_TDATA  (tdata),
56      .AXI4_Stream_Master_TVALID (tvalid),
57      .AXI4_Stream_Master_TLAST  (tlast)
58  );
```

Lines 32–58: The device under test

The HDL Coder IP core is a VHDL entity instantiated from SystemVerilog; xsim handles the mixed languages [8]. Port names are the IP's (`AXI4_Lite_*`, `AXI4_Stream_Master_*`) [11]. Two details:

- Both clocks (`IPCORE_CLK`, `AXI4_Lite_ACLK`) and both resets come from the same `clk`/`rstn`, as they will in the block design.
- `AXI4_Lite_WSTRB` is tied to `4'hF`: every write updates all four bytes of the 32-bit register [1].

```
59
60  // ------------------------------------------------------------ AXI4
      -Lite
61  task automatic axi_write(input logic [15:0] addr, input logic [31:0]
      data);
62    @(posedge clk);
63    awaddr <= addr; awvalid <= 1; wdata <= data; wvalid <= 1; bready
     <= 1;
64    fork
65      begin do @(posedge clk); while (!awready); awvalid <= 0; end
66      begin do @(posedge clk); while (!wready);  wvalid  <= 0; end
67    join
68    do @(posedge clk); while (!bvalid);
69    bready <= 0;
70    if (bresp != 0) $error("write 0x%04h: BRESP=%0d", addr, bresp);
71  endtask
```

Lines 59–71: axi_write: one AXI4-Lite write, handshake by handshake

- **Line 62**: start on a clock edge. After `@(posedge clk)` returns, signals read have their values from just before the edge, the value a flip-flop captures [3].
- **Line 63**: present address and data and raise AWVALID and WVALID together, without looking at READY (**rule 2**: VALID must not depend on READY, or master and slave can deadlock [1, 6]). BREADY goes high early (**rule 4**: READY may precede VALID). `<=` is nonblocking: the IP sees the new values at the next edge.

- **Lines 64–67**: `fork/join` waits on the AW and W channels in parallel [5] (**rule 5**: they are independent and either may finish first). Each branch checks its READY at every edge and drops its own VALID only after the handshake edge (**rule 3**: VALID and data hold until the transfer; **rule 1**: a transfer is VALID and READY high at the same edge). `awaddr` and `wdata` are never touched while waiting.
- **Lines 68–69**: a write is complete only when the B response arrives (**rule 6**).
- **Line 70**: BRESP must be 0, OKAY (**rule 8**); anything else is reported with the address.

In the run: packet-size write AW at edge 24, W at edge 25 (WREADY was low at 24, so WVALID and the data held one cycle), B at edge 26.

```
72
73  task automatic axi_read(input logic [15:0] addr, output logic [31:0]
      data);
74    @(posedge clk);
75    araddr <= addr; arvalid <= 1; rready <= 1;
76    do @(posedge clk); while (!arready);
77    arvalid <= 0;
78    do @(posedge clk); while (!rvalid);
79    data = rdata;
80    rready <= 0;
81    if (rresp != 0) $error("read 0x%04h: RRESP=%0d", addr, rresp);
82  endtask
```

Lines 72–82: axi_read: address, then data

- **Line 75**: raise ARVALID with the address (**rule 2**) and RREADY in advance (**rule 4**).
- **Lines 76–77**: wait for the AR handshake, then drop ARVALID (**rules 1, 3**).
- **Lines 78–79**: wait for RVALID and capture RDATA on that edge (**rule 7**: a read is an AR handshake followed by an R handshake [1]). The blocking `=` takes the value immediately.
- **Lines 80–81**: drop RREADY and check RRESP (**rule 8**).

In the run: AR at edge 21 (the IP's ARREADY was already high), R at edge 22 with RDATA = 2610121705, the IP build timestamp (yymmddHHMM).

```
83
84  // ------------------------------------------------------------ stream
      sink
85  int    fd, beats = 0, cycle = 0;
86  string fname;
87
88  always @(posedge clk) begin
89    cycle <= cycle + 1;
90    if (tvalid && tready && beats < N_FRAMES * FRAME) begin
91      $fdisplay(fd, "%0d %0d", tdata, tlast);
```

```
92        beats <= beats + 1;
93      end
94      // ~75% random TREADY, and a 1000-cycle stall to fill the IP
        output FIFO
95      if (!rstn || (cycle >= 4000 && cycle < 5000)) tready <= 0;
96      else                                          tready <= (
        $urandom_range(3) != 0);
97    end
```

Lines 83–97: The AXI4-Stream receiver

- **Line 88**: evaluated at every rising edge.
- **Line 90**: a beat counts only when TVALID and TREADY were both high at this edge (**rule 1**; **rule 9**: TDATA means nothing on other edges [2]). Logging stops at exactly 2048 beats.
- **Line 91**: TDATA and TLAST are written together, one line per transfer (**rule 10**: TLAST belongs to its beat).
- **Lines 95–96**: the receiver's READY policy (**rule 4**: a destination may withhold READY for as long as it likes). TREADY is low in reset and for cycles 4000–4999, otherwise high on a random 3 of 4 edges (`$urandom_range(3)` returns 0–3 [3]). TVALID never enters the decision, so READY is independent of VALID.

The randomness and the long stall push the IP through its flow-control path: its output FIFO fills, its auto-ready signal drops and the signal chain's clock enable pauses [11]. *In the run:* TREADY high 74.7 % of the time while streaming; 1,697 edges where the IP held a sample with TREADY low, none logged twice; chain frozen from edge 4005 to 5006.

```
98
99    // ------------------------------------------------------------------
        main
100   logic [31:0] r;
101
102   initial begin
103     if (!$value$plusargs("out=%s", fname)) fname = "ip_stream.txt";
104     fd = $fopen(fname, "w");
105
106     repeat (10) @(posedge clk);
107     rstn <= 1;
108     repeat (10) @(posedge clk);
109
110     axi_read(REG_TIMESTAMP, r);
111     $display("IP timestamp: %0d", r);              // yymmddHHMM
112
113     // packet size is write-only; set it like the software driver will
114     axi_write(REG_PACKET_SIZE, FRAME);
115
116     // start the signal chain (enable -> NCO validIn)
117     axi_write(REG_ENABLE, 1);
118
```

```
119     wait (beats == N_FRAMES * FRAME);
120     repeat (10) @(posedge clk);
121     $fclose(fd);
122     $display("IP TB DONE: %0d beats, %0d cycles", beats, cycle);
123     $finish;
124   end
```

Lines 98–124: The test sequence

- **Lines 103–104**: the output file comes from the `+out=` plusarg that step 10's Tcl passes to xsim [8], so results land in `results/vivado/ip_tb`.
- **Lines 106–108**: 10 cycles in reset, release, then 10 idle cycles so the IP's reset synchroniser has released (**rule 11**). Reset was released at edge 10.
- **Lines 110–111**: read the timestamp first: a harmless read that proves the AXI4-Lite path works before any IP state changes.
- **Line 114**: set the packet size to 1024 beats. The register is write-only, so it is set the way the software driver will set it.
- **Line 117**: write 1 to `enable`, which drives the NCO's `validIn`: the signal chain starts. Because `axi_write` waits for each B response, the packet size is guaranteed to be in place first (**rule 6**).
- **Lines 119–123**: wait for 2048 beats, close the file and finish. Step 11 then compares the file with the cosimulation and checks the TLAST positions.

```
125
126   initial begin
127     #1ms;
128     $display("IP TB TIMEOUT: %0d beats", beats);
129     $fclose(fd);
130     $finish;
131   end
132
133 endmodule
```

Lines 125–133: Watchdog

A second `initial` block runs in parallel with the main one. If 1 ms of simulated time passes (125,000 cycles; the real run needs 5931), it reports how many beats arrived and ends the simulation, so a hung handshake can never stall the run forever.

Results

Run with Vivado 2024.1 xsim through `pipeline/step10_run_ip_tb.m` and checked by `step11_check_ip_tb.m`: 5931 cycles; 2048 beats; 0 mismatches against the cosimulation; TLAST on beats 1024 (edge 3546) and 2048 (edge 5921) only; all register responses OKAY.

Limitations

The testbench obeys every rule as the master and as the stream receiver and checks the data. It does not assert the matching obligations on the IP's own outputs, for example that TVALID, TDATA and TLAST stay stable while TREADY is low. A violation would most likely appear as a data mismatch, but would not be reported as a protocol error. SystemVerilog assertions [3] or the protocol checker in the AMD AXI Verification IP [9] would close that gap.

References

[1] Arm Ltd., *AMBA AXI Protocol Specification*, IHI 0022 (issue L). https://developer.arm.com/documentation/ihi0022/latest

[2] Arm Ltd., *AMBA AXI-Stream Protocol Specification*, IHI 0051 (issue B). https://developer.arm.com/documentation/ihi0051/latest

[3] IEEE, *IEEE Standard for SystemVerilog: Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2023. https://standards.ieee.org/ieee/1800/7743/

[4] ChipVerify, *SystemVerilog Tutorial*. https://www.chipverify.com/systemverilog/systemverilog-tutorial

[5] ChipVerify, *SystemVerilog fork join*. https://www.chipverify.com/systemverilog/systemverilog-fork-join

[6] D. Gisselquist, "AXI Handshaking Rules," *ZipCPU*, Aug. 2021. https://zipcpu.com/blog/2021/08/28/axi-rules.html

[7] D. Gisselquist, "Building an AXI-Lite slave the easy way," *ZipCPU*, Mar. 2020. https://zipcpu.com/blog/2020/03/08/easyaxil.html

[8] AMD, *Vivado Design Suite User Guide: Logic Simulation*, UG900. https://docs.amd.com/r/en-US/ug900-vivado-logic-simulation

[9] AMD, *AXI Verification IP LogiCORE IP Product Guide*, PG267. https://docs.amd.com/r/en-US/pg267-axi-vip

[10] MathWorks, *Custom IP Core Generation*, HDL Coder documentation. https://www.mathworks.com/help/hdlcoder/ug/custom-ip-core-generation.html

[11] MathWorks, *Model Design for AXI4-Stream Interface Generation*, HDL Coder documentation. https://www.mathworks.com/help/hdlcoder/ug/model-design-for-axi4-stream-interface-generation.html

[12] MathWorks, *AXI Manager Simulation*, HDL Coder documentation. https://www.mathworks.com/help/hdlcoder/alterasochdlcoder/ug/matlab-axi-master-simulation.html